Laboratorio 04

DOCENTE	CARRERA	CURSO
DSc. Edgar Sarmiento Calisaya	Escuela Profesional de Ciencia de la Computación	Ingeniería de Software II

LABORATORIO	TEMA	DURACIÓN
04	xUnit y Cobertura de Código	2 horas

9. Competencias del curso

- Ejecuta pruebas estáticas y dinámicas de componentes de software mediante el uso de herramientas y técnicas de caja blanca y caja negra en colaboración con los clientes.
- Trabaja individualmente y en equipo, de manera ética y con compromiso, para el desarrollo de soluciones.

10. Competencias de la práctica

- Implementa casos de prueba utilizando el framework xUnit y cobertura de código.

11. Equipos y materiales

- Computadora
- Internet
- Material específico del curso descrito en el aula virtual:

12. Objetivo

- Aprender los conceptos de pruebas unitarias utilizando el framework xUnit y cobertura de código.

13. Prerrequisito

- IDE: plugin/extension ***xUnit*** y Cobertura de Código (***Code Coverage)***
- Práctica 5: Un desarrollador creó un "programa que lee una lista de valores enteros y los ordena en forma ascendente":

```
bubbleSort(int vet[]){
  integer i,j,aux;
  for (i=tamanho(vet); i >= 2; i--){
     for (j=1; j <= i-1;j++){
        if (vet[j] > vet[j+1]){
           aux = vet[j];
           vet[j] = vet[j+1];
           vet[j+1] = aux;
        }
     }
  }
}
```

14. Actividades

1. Construir el grafo de flujo de control correspondiente a este código.
2. Calcular la complejidad ciclomática
3. Identificar los **caminos de ejecución independientes** con base en una estrategia de cobertura: **cobertura de decisiones + condición simple o condición múltiple.**
4. Diseñar un caso de prueba para cada camino de ejecución:

ID CP	Camino en el Grafo	Valores de Prueba	Resultado Esperado
CP1			
CP2			
CP3			
CP4			

5. En su IDE, crear un proyecto en el lenguaje de programación de su preferencia (C++, C#, Java o Python) y el paradigma orientado a objetos.
6. Seleccionar el framework ***xUnit*** (p.ejm. GTest, MSTest/NUnit, JUnit/TestNG o Unittest/PyUnit) correspondiente al lenguaje de programación escogido: Importarlo en su proyecto vía su gestor de dependencias o sistema de construcción automática. Por ejemplo, en Java con Maven:

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-ins
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.mycompany.app</groupId>
    <artifactId>junitmavenexample</artifactId>
    <packaging>jar</packaging>
    <version>1.0-SNAPSHOT</version>
    <name>junitmavenexample</name>
    <url>http://maven.apache.org</url>
    <properties>
        <maven.compiler.source>1.8</maven.compiler.source>
        <maven.compiler.target>1.8</maven.compiler.target>
    </properties>

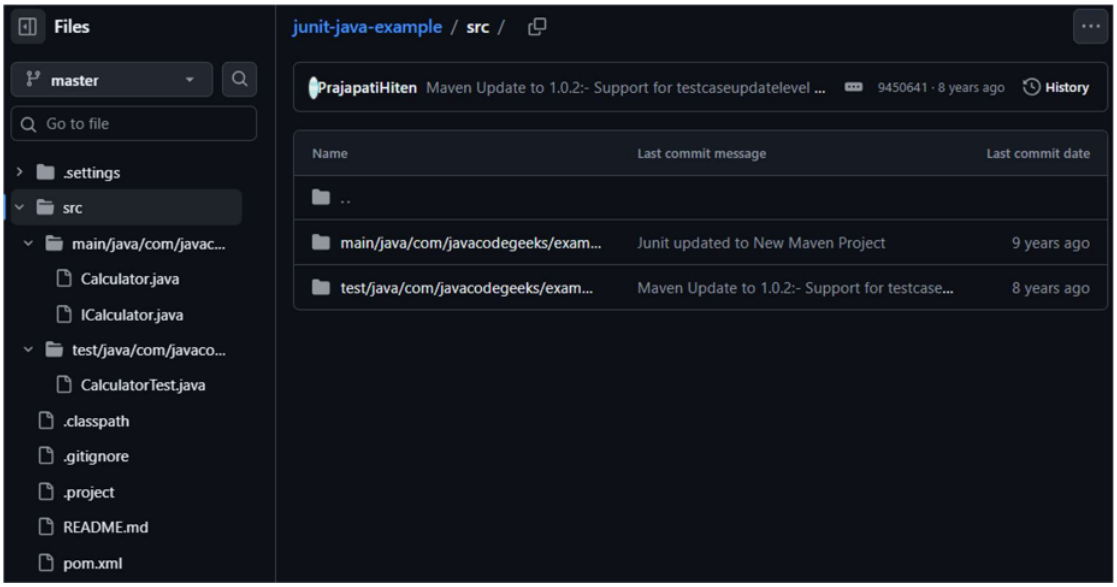
    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.11</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

```

7. Seleccionar una herramienta para generar el reporte de cobertura de código – ***Code Coverage*** (p.ejm. Gcov, Coverlet, JaCoCo/Emma o Coverage.py) de las pruebas implementadas vía ***xUnit***, correspondiente al lenguaje de programación: Importarla como biblioteca (lib) en su proyecto vía su gestor de dependencias o sistema de

construcción automática.

8. Construir su proyecto: Separar el código de implementación de las pruebas (p.ejm. main y test)



1. Implementar el método **bubbleSort**(int vet[]). Por ejemplo, en Java con Maven:

```
package com.javacodegeeks.examples.junitmavenexample;

public interface ICalculator {
        int add(int a, int b);

        int subtract(int a, int b);

        int divide(int a, int b) throws Exception;
}
```

```
package com.javacodegeeks.examples.junitmavenexample;

public class Calculator implements ICalculator {
  @Override
   public int add(int a, int b) {
      return a + b;
   }

  @Override
   public int subtract(int a, int b) {
      return a - b;
   }

  @Override
   public int divide(int a, int b) {
      if (b == 0) {
         throw new IllegalArgumentException("Division by zero is not allowed");
      }
      return a / b;
   }
}
```

2. Probar el programa por medio de **3 casos de prueba** (complejidad ciclomática), y automatizarlo usando métodos o macros **xUnit** (***Test Suites, Test Cases, Assertions, SetUp y TearDown***):

   a. Especificar el caso de prueba

Camino en el Grafo	Valores de Prueba	Resultado Esperado

   b. Implementar el caso de prueba: Inicializar (**Arrange**), Ejecutar la Prueba (**Act/Execute**) y Verificar resultado (**Verify**). Por ejemplo, en Java con JUnit 5:

```
package com.javacodegeeks.examples.junitmavenexample;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.DisplayName;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

class CalculatorTest {

    private ICalculator calculator;

    @BeforeEach
    void setUp() {
        calculator = new Calculator(); // Initialize the SUT
    }

    @Test
    @DisplayName("Test addition of two numbers")
    void testAdd() {
        // Arrange
        int num1 = 5;
        int num2 = 3;
        int expectedSum = 8;

         // Act
        int actualSum = calculator.sum(num1, num2); // Execute the method under
test

        // Assert
        assertEquals(expectedSum, actualSum, "The sum of two numbers should be
correct."); // Verify the result
    }

    @Test
    @DisplayName("Test division by zero throws exception")
    void testDivideByZero() {
        // Arrange
       int num1 = 10;
       int num2 = 0;

      // Act & Verify
        assertThrows(IllegalArgumentException.class, () -> {
            calculator.divide(num1, num2);
        }, "Division by zero should throw IllegalArgumentException");
    }
```

c. Compilar la implementación.

d. Ejecutar la prueba y observe el resultado: PASÓ/FALLÓ

```
mvn test
```

e. En caso de FALLO, ACTUALIZAR el código lo suficiente para que la prueba pase

   i. Volver a ejecutar la prueba y vea cómo pasa.

f. Refactorizar para mayor claridad y "sólo una vez"

g. Repetir hasta tener 3 casos de prueba

3. Generar el reporte de ejecución de los casos de prueba (¿Pasó o Falló?): Por terminal o extensión de su *framework* en su *IDE*.
4. Generar el reporte de cobertura de código (cantidad de código ejecutado por los casos de prueba): Por terminal o extensión de su *framework* en su *IDE*.

15. Entregables

- Especificación de Casos de Prueba: Diagrama de Flujo de Control (o flujograma), Complejidad Ciclomática, y Casos de Prueba identificados (formato tabular)

ID CP	Camino en el Grafo	Valores de Prueba	Resultado Esperado
CP1			
CP2			
CP3			
CP4			

- Implementación de Casos de Prueba: Proyecto en **GitHub**
- Reporte de ejecución de pruebas: HTML o captura de pantalla
- Reporte de cobertura de código: HTML o captura de pantalla

16. Conceptos y Documentación

- Material sobre Whitebox Testing
  - https://miro.com/diagramming/data-flow-diagram-vs-flowchart/
  - https://www.geeksforgeeks.org/difference-between-flowchart-and-data-flow-diagram-dfd/
- Ejemplo con Junit:
  - https://github.com/qmetry/junit-java-example
- xUnit: Implementaciones por Lenguaje de Programación

    - https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-with-mstest
    - https://www.baeldung.com/junit-5
    - https://docs.python.org/3/library/unittest.html
    - https://saucelabs.com/resources/blog/jest-vs-mocha
- Code Coverage Tools:
    - https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-code-coverage?tabs=windows
    - https://www.baeldung.com/jacoco
    - https://coverage.readthedocs.io/en/7.6.4/